

# LAPORAN PRAKTIKUM

## Keamanan Backend pada Sistem Terdistribusi

### Implementasi JWT, bcrypt, dan AES-256-GCM

Mata kuliah: Sistem Terdistribusi

Tugas: Tugas 2 - Tantangan Sistem Terdistribusi

Nama: [Isi nama mahasiswa]

NIM: [Isi NIM]

Kelas: [Isi kelas]

Program studi: Informatika

Dosen pengampu: [Isi nama dosen]

Repositori GitHub: [Isi URL setelah kode diunggah]

### Status dokumen

Implementasi backend dan pengujian otomatis telah disiapkan. Dokumen ini masih berupa draf untuk dilengkapi dengan identitas, tautan repositori, dan screenshot pengujian Postman asli. Hasil pengujian otomatis tidak dinyatakan sebagai hasil pengujian melalui aplikasi Postman.

### Ringkasan

Praktikum ini mengimplementasikan dua endpoint backend menggunakan Node.js dan Express. Endpoint login memeriksa kata sandi dengan bcrypt dan menghasilkan JWT. Endpoint data terproteksi memverifikasi JWT sebelum mengirimkan data sensitif dalam bentuk ciphertext AES-256-GCM. Seluruh data pengguna pada aplikasi merupakan data simulasi.

# 1. Pendahuluan dan Tujuan

Komponen sistem terdistribusi berkomunikasi melalui jaringan sehingga identitas pengakses dan kerahasiaan data perlu diperhatikan. Pada praktikum ini, perlindungan diterapkan pada sisi backend agar data hanya dapat diminta oleh pengguna yang memiliki token valid. Implementasi masih menggunakan satu proses backend lokal dan satu klien; fokusnya adalah mekanisme keamanan, bukan pengujian cluster multi-node.

Tujuan praktikum adalah membuat service sederhana, menerapkan autentikasi JWT, menyimpan representasi kata sandi sebagai hash bcrypt, mengenkripsi data sensitif pada payload respons, serta menguji penolakan dan pemberian akses.

## 1.1 Kesesuaian dengan instruksi tugas

- Persiapan service: Node.js dan Express menyediakan POST /login serta GET /protected-data.
- JWT: token dibuat setelah login benar, lalu diperiksa oleh middleware pada endpoint terproteksi.
- Password hashing: hash bcrypt cost 12 dibuat saat setup; login menggunakan bcrypt.compare.
- Enkripsi simetris: respons data sensitif menggunakan AES-256-GCM.
- Pengujian: disediakan koleksi Postman untuk tanpa token, token invalid, dan token valid, ditambah dua skenario login.
- Penyerahan: kode siap diunggah ke GitHub; PDF harus dilengkapi screenshot asli sebelum dikumpulkan.

## 1.2 Perangkat dan struktur

Perangkat lunak yang digunakan adalah Node.js 22 atau lebih baru, npm, Express, bcrypt, jsonwebtoken, express-rate-limit, dan Postman. Modul crypto bawaan Node.js menangani enkripsi dan dekripsi. Tidak diperlukan database karena tugas ini menggunakan satu akun demonstrasi; hash berada pada konfigurasi lokal dan dibaca ke memori saat service dimulai.

File src/app.js berisi route dan middleware; src/config.js memvalidasi konfigurasi; src/encryption.js menangani kriptografi; src/server.js menjalankan service. Folder scripts menyediakan setup dan demo, sedangkan folder test dan postman berisi perangkat pengujian.

## 2. Perancangan dan Implementasi

### 2.1 Alur autentikasi

Klien mengirim username dan password ke POST /login. Server memvalidasi tipe dan panjang input, kemudian menjalankan `bcrypt.compare` terhadap hash. Jika salah, server mengembalikan HTTP 401 tanpa mengungkapkan apakah username atau password yang salah. Jika benar, server menandatangani JWT HS256 dengan masa berlaku 15 menit.

JWT berisi username, subject, issuer, audience, waktu terbit, dan waktu kedaluwarsa. JWT ini ditandatangani, bukan dienkripsi. Oleh sebab itu password dan kunci enkripsi tidak dimasukkan ke payload token. Middleware membatasi algoritma HS256 serta memeriksa signature, issuer, audience, subject, identitas akun, dan expiry sebelum meneruskan permintaan (Auth0, n.d.).

Klien menyertakan Authorization: Bearer <token> pada GET /protected-data. Token hilang, format header salah, signature tidak sah, atau token kedaluwarsa menghasilkan HTTP 401. Token valid menghasilkan HTTP 200 dan data terenkripsi. HTTP 401 digunakan untuk kegagalan autentikasi; aplikasi ini belum memiliki skenario perbedaan peran yang membutuhkan HTTP 403.

### 2.2 Hashing kata sandi

Setup menghasilkan hash `bcrypt cost 12` dengan salt yang dikelola oleh pustaka. Saat login, password yang dikirim pengguna diperiksa memakai `bcrypt.compare`, bukan membandingkan dua hasil hash baru. Password dibatasi maksimal 72 byte agar tidak terkena pemotongan input `bcrypt` secara diam-diam (Kelektiv, n.d.).

Password demo sengaja tercantum pada petunjuk dan skrip setup agar praktikum dapat direproduksi. Aplikasi tidak menyimpan password plaintext di `.env`. Akun demo ini tidak boleh digunakan untuk layanan produksi.

### 2.3 Enkripsi payload

Data simulasi dienkripsi memakai AES-256-GCM, kunci 32 byte, IV acak 12 byte untuk setiap enkripsi, dan authentication tag 16 byte. Respons memuat algorithm, encoding, iv, authTag, dan ciphertext dalam Base64; kunci tidak dikirimkan. GCM memungkinkan dekripsi mendeteksi perubahan ciphertext atau tag (Node.js, n.d.).

Skrip demo mendekripsi respons secara lokal menggunakan kunci konfigurasi yang sama. Ini hanya simulasi klien tepercaya pada mesin yang sama. Pendistribusian kunci antar mesin belum dibuat. Enkripsi payload tidak menggantikan HTTPS untuk perlindungan kredensial dan token selama transmisi.

## 3. Pelaksanaan dan Panduan Postman

### 3.1 Menjalankan backend

Ekstrak proyek dan buka terminal pada folder yang berisi package.json. Jalankan perintah berikut secara berurutan. Node.js dan npm harus sudah terpasang.

```
npm install
npm run setup
npm test
npm start
```

Server aktif pada `http://127.0.0.1:3000`. Akun latihan adalah username mahasiswa dengan password `Praktikum@2026`. Setup tidak menerima `.env` yang sudah ada. Untuk PowerShell yang memblokir `npm.ps1`, gunakan `npm.cmd` sebagai pengganti `npm`.

### 3.2 Urutan pengujian

Import `postman/Tugas2.postman_collection.json` ke Postman desktop dan jalankan request 01 sampai 05. Pada request login berhasil, skrip Postman otomatis menyimpan token dalam variabel koleksi. Jika token sudah kedaluwarsa, jalankan login kembali.

- Skenario 01: GET `/protected-data` tanpa Authorization. Hasil yang diharapkan HTTP 401 dengan pesan Token diperlukan.
- Skenario 02: GET `/protected-data` dengan Bearer `invalid.token.demo`. Hasil yang diharapkan HTTP 401 dengan pesan token tidak valid atau kedaluwarsa.
- Skenario 03: POST `/login` dengan username benar dan password salah. Hasil yang diharapkan HTTP 401.
- Skenario 04: POST `/login` dengan akun latihan yang benar. Hasil yang diharapkan HTTP 200 dan token JWT.
- Skenario 05: GET `/protected-data` dengan token hasil skenario 04. Hasil yang diharapkan HTTP 200 dan objek `encryptedData`.

### 3.3 Bukti yang perlu direkam

Setiap screenshot harus memperlihatkan metode, URL, status HTTP, dan body respons. Tampilkan konfigurasi Authorization bila diperlukan untuk membedakan skenario. Gunakan nama gambar sesuai petunjuk lampiran. Jangan menampilkan JWT secret, kunci AES, atau isi `.env`. Token akses boleh disamarkan saat screenshot dibagikan.

Untuk pembuktian enkripsi tambahan, jalankan `npm run demo` pada terminal kedua sementara server masih berjalan. Skrip menampilkan ciphertext dan hasil dekripsi lokal berupa data simulasi. Demonstrasi ini tidak menggantikan tiga pengujian wajib Postman.

## 4. Hasil Pengujian dan Analisis

### 4.1 Hasil otomatis yang telah diverifikasi

Pengujian otomatis dijalankan pada lingkungan pengerjaan dengan Node.js 24.19.0. Lima belas pengujian lulus. Keluaran asli tersimpan dalam docs/Hasil\_Uji\_Otomatis.txt. Pengujian HTTP menggunakan server lokal pada port sementara dan data uji sintetis, bukan aplikasi Postman.

- Hash bcrypt berbeda dari plaintext dan password yang benar cocok.
- Tanpa token dan token invalid ditolak dengan HTTP 401.
- Password salah ditolak dengan HTTP 401; input kosong ditolak dengan HTTP 400.
- Login valid menghasilkan JWT dengan masa berlaku 900 detik.
- Token valid mendapat HTTP 200 dengan ciphertext yang dapat didekripsi kembali.
- Token kedaluwarsa, signature salah, dan audience salah ditolak dengan HTTP 401.
- IV/ciphertext berubah antar enkripsi; perubahan ciphertext dan kunci salah menggagalkan dekripsi.
- Konfigurasi kosong ditolak saat pemuatan; malformed JSON dan password terlalu panjang menghasilkan HTTP 400.
- Percobaan login melampaui batas pengujian menghasilkan HTTP 429.

### 4.2 Status pengujian Postman

Belum dilaksanakan melalui antarmuka Postman pada lingkungan pengerjaan ini. Hasil pada bagian 3 adalah hasil yang diharapkan, bukan klaim hasil pengamatan Postman. Bagian ini harus diperbarui setelah mahasiswa menjalankan koleksi dan melampirkan screenshot asli.

### 4.3 Analisis keamanan dan keterbatasan

Penolakan token hilang dan invalid menunjukkan bahwa middleware menghalangi akses tanpa autentikasi yang sah. Pengujian login dan expiry memastikan token tidak diterbitkan untuk password salah dan tidak berlaku tanpa batas. Hashing mengurangi risiko penyimpanan password plaintext, tetapi tidak menghilangkan kebutuhan password kuat dan perlindungan konfigurasi.

Ciphertext yang berubah menunjukkan penggunaan IV baru; round-trip serta penolakan ciphertext yang dimodifikasi menguji fungsi enkripsi dan integritas. Hasil ini bukan audit keamanan menyeluruh. HTTP loopback digunakan hanya untuk praktikum. Produksi membutuhkan HTTPS, pengelolaan serta rotasi kunci, mekanisme pencabutan token, pemantauan, dan pengujian lanjutan.

Dalam penerapan multi-instance, konfigurasi verifikasi JWT harus konsisten antar layanan dan rahasia tetap dikelola secara aman. Rate limiter berbasis memori pada satu proses belum membatasi seluruh cluster; perlu penyimpanan bersama. Skema distribusi kunci AES untuk klien jarak jauh juga belum termasuk lingkup implementasi.

## 5. Kesimpulan dan Kelengkapan Penyerahan

Backend telah mengimplementasikan login dengan bcrypt, penerbitan dan verifikasi JWT, serta enkripsi data sensitif dengan AES-256-GCM. Lima belas pengujian otomatis lulus. Sebelum penyerahan, pengujian Postman harus dijalankan, screenshot asli ditambahkan, dan kode diunggah ke repositori GitHub mahasiswa.

### 5.1 Daftar periksa

- Isi nama, NIM, kelas, serta dosen pada halaman pertama.
- Jalankan backend di laptop dan lakukan lima skenario Postman.
- Lengkapi lampiran screenshot dan perbarui status pengujian Postman.
- Unggah kode dan laporan ke GitHub tanpa .env dan node\_modules.
- Cantumkan URL repositori pada halaman pertama.
- Bangun ulang atau ekspor PDF final; periksa semua halaman sebelum dikumpulkan.

### 5.2 Keterangan gambar yang digunakan

Gambar 1. Penolakan akses endpoint terproteksi tanpa token JWT. Simpan sebagai docs/screenshots/01-tanpa-token.png.

Gambar 2. Penolakan akses menggunakan token JWT tidak valid. Simpan sebagai docs/screenshots/02-token-invalid.png.

Gambar 3. Penolakan login karena kata sandi salah. Simpan sebagai docs/screenshots/03-password-salah.png.

Gambar 4. Login berhasil dan penerbitan token JWT. Simpan sebagai docs/screenshots/04-login-berhasil.png.

Gambar 5. Akses menggunakan token valid dan respons data terenkripsi AES-256-GCM. Simpan sebagai docs/screenshots/05-token-valid.png.

Skip scripts/build\_report.py otomatis menambahkan gambar yang tersedia sebagai lampiran. Jika tidak ada gambar, tidak dibuat bukti pengujian pengganti.

### Daftar pustaka

Auth0. (n.d.). node-jsonwebtoken [Perangkat lunak]. GitHub. <https://github.com/auth0/node-jsonwebtoken>

Kelektiv. (n.d.). node.bcrypt.js [Perangkat lunak]. GitHub. <https://github.com/kelektiv/node.bcrypt.js>

Node.js. (n.d.). Crypto. Node.js documentation. <https://nodejs.org/api/crypto.html>

Tantangan sistem terdistribusi. (2025). [Materi perkuliahan, slide 15: Tugas 2]. Dokumen yang diberikan pada tugas.